

5. Semana 5. Simulación Bidimensional de Campos

Objetivo de la cátedra

Implementar simulaciones bidimensionales de difusión, propagación de ondas y transporte en dominios con geometría compleja, dominando el mallado cartesiano 2D, la representación de obstáculos y cavidades mediante máscaras, el ensamblaje disperso de sistemas lineales, los criterios de estabilidad en dos dimensiones y la visualización cuantitativa de campos escalares, con énfasis en la interpretación física y el diagnóstico de los errores numéricos propios de 2D.

Temario

- Simulación 2D de difusión.
- Propagación de ondas.
- Obstáculos y condiciones geométricas.
- Mallado bidimensional.
- Visualización de campos escalares.
- Análisis temporal de simulaciones.
- Interpretación física de resultados.

Propósito del tema. La Semana 4 estableció las EDPs canónicas y sus esquemas en una dimensión espacial. Esta semana da el salto a dos dimensiones, que es donde la simulación empieza a parecerse a los problemas reales de una tesis doctoral: secciones transversales de componentes compuestos, placas con cavidades, recintos con obstáculos, flujos recirculantes y campos medidos por redes de sensores. El salto no es trivial: el costo computacional crece cuadráticamente con la resolución, los criterios de estabilidad se endurecen, la geometría deja de ser un intervalo y se convierte en una decisión de modelado, y la elección de condiciones de frontera en los bordes *internos* (huecos, obstáculos) puede invertir por completo la conclusión de un estudio. Esta semana se aprende a anticipar y diagnosticar esos efectos.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) discretizar el laplaciano 2D con el stencil de cinco puntos y cuantificar su error de truncamiento; (ii) derivar y aplicar los criterios de estabilidad FTCS y CFL en dos dimensiones; (iii) representar geometrías complejas (dominios circulares, perforaciones, obstáculos) sobre mallas cartesianas mediante máscaras booleanas, incluyendo interfaces entre materiales con media armónica;

(iv) ensamblar y resolver sistemas dispersos para problemas elípticos en dominios no triviales; (v) cuantificar la difusión y la dispersión numéricas de los esquemas de transporte en 2D mediante pruebas con solución exacta conocida; y (vi) seleccionar visualizaciones de campos escalares que permitan comparación cuantitativa y no solo cualitativa.

5.1. Motivación: del eje 1D al campo 2D

En una dimensión, la incógnita $u(x, t)$ vive sobre una línea y la geometría se reduce a un intervalo con dos extremos. En dos dimensiones, $u(x, y, t)$ vive sobre una región $\Omega \subset \mathbb{R}^2$ cuya *forma* es parte del modelo. Esto introduce cuatro diferencias estructurales que dominan el trabajo de esta semana:

1. **Costo computacional.** Una malla de N nodos por dirección tiene N^2 incógnitas. Duplicar la resolución multiplica por 4 la memoria y, en esquemas explícitos cuyo paso temporal escala como $\Delta t \propto \Delta x^2$, multiplica por 8 el número total de operaciones de una simulación transitoria de difusión.
2. **Geometría.** Los dominios reales son círculos, placas perforadas, recintos con obstáculos. Sobre una malla cartesiana, la herramienta básica para representarlos es la *máscara booleana*: un arreglo del mismo tamaño que la malla que indica qué celdas pertenecen al dominio físico.
3. **Condiciones de frontera internas.** Además del contorno exterior, aparecen fronteras *interiores*: paredes de cavidades, superficies de obstáculos, interfaces entre materiales. La elección de condición en esas fronteras (Dirichlet, Neumann, Robin) tiene consecuencias físicas de primer orden, como se demostrará cuantitativamente en el Ejemplo 3.
4. **Visualización.** En 1D basta una curva $u(x)$; en 2D el resultado es un campo escalar que requiere mapas de color, contornos y cortes, y cuya lectura incorrecta (escalas de color distintas entre paneles, mapas no perceptualmente uniformes) puede inducir conclusiones falsas.

Los seis ejemplos de esta semana cubren las tres familias de EDPs de la Semana 4, ahora en 2D, con situaciones representativas de los proyectos del grupo (sin particularizar): el calentamiento transitorio de una *sección transversal compuesta* con generación interna de calor; la *propagación y difracción de ondas* ante un obstáculo; el campo estacionario de temperatura en una *placa con cavidades*; el *transporte de un trazador en un flujo recirculante*; la *advección de un campo de densidad* bajo un campo de velocidad conocido punto a punto; y la *reconstrucción de un campo* a partir de mediciones dispersas de una red de sensores.

5.2. Mallado bidimensional y discretización del laplaciano

5.2.1. Malla cartesiana estructurada

Sea $\Omega = [0, L_x] \times [0, L_y]$ y una malla uniforme de $N_x \times N_y$ nodos:

$$x_i = i \Delta x, \quad y_j = j \Delta y, \quad \Delta x = \frac{L_x}{N_x - 1}, \quad \Delta y = \frac{L_y}{N_y - 1}, \quad (152)$$

con $u_{i,j}^n \approx u(x_i, y_j, t_n)$. En NumPy esta malla se construye con `meshgrid`, y la convención de indexación (`indexing='xy'` o `'ij'`) debe declararse explícitamente y mantenerse coherente en todo el código: una fracción considerable de los errores en simulaciones 2D proviene de confundir el eje de filas con el eje y físico.

5.2.2. El stencil de cinco puntos

Aplicando la diferencia centrada de segundo orden (Semana 3) en cada dirección, el laplaciano discreto con $\Delta x = \Delta y = h$ es

$$\nabla^2 u|_{i,j} \approx \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2}, \quad (153)$$

con error de truncamiento

$$\tau_{i,j} = \frac{h^2}{12} \left(\frac{\partial^4 u}{\partial x^4} + \frac{\partial^4 u}{\partial y^4} \right) + \mathcal{O}(h^4), \quad (154)$$

es decir, segundo orden espacial, igual que en 1D. La novedad no está en el orden sino en la *estructura algebraica*: al ordenar las incógnitas lexicográficamente ($p = jN_x + i$), el operador (153) genera una matriz pentadiagonal de tamaño $N^2 \times N^2$ con a lo sumo cinco entradas no nulas por fila.

5.2.3. Por qué el almacenamiento disperso es obligatorio

Tabla 22: Costo de memoria del laplaciano 2D para malla $N \times N$ (entradas en doble precisión, 8 bytes).

N	Incógnitas N^2	Matriz densa	Matriz dispersa
51	2 601	54 MB	0.1 MB
181	32 761	8.6 GB	1.3 MB
501	251 001	504 GB	10 MB
1001	1 002 001	8000 GB	40 MB

Para $N = 181$ (la malla del Ejemplo 3) la matriz densa ya no cabe en la memoria de una sesión estándar de Google Colab, mientras que la versión dispersa ocupa poco más de un megabyte. El módulo `scipy.sparse` provee los formatos `lil_matrix` (eficiente para

ensamblar entrada por entrada) y `csr_matrix` (eficiente para resolver con `spsolve`); el flujo de trabajo estándar es ensamblar en LIL y convertir a CSR antes de resolver.

5.3. Estabilidad en dos dimensiones

Los criterios de la Semana 3 se endurecen en 2D porque el error puede crecer simultáneamente por ambas direcciones. El análisis de von Neumann con modos $e^{i(k_x x + k_y y)}$ produce:

- **Difusión, FTCS 2D.** Con $\Delta x = \Delta y$, la condición $F_{o,x} + F_{o,y} \leq \frac{1}{2}$ se reduce a

$$\Delta t \leq \frac{\Delta x^2}{4\alpha}, \quad (155)$$

exactamente la mitad del límite 1D. Con propiedades variables, α debe evaluarse en su valor *máximo* sobre el dominio.

- **Onda, leapfrog 2D.** El número de Courant admisible disminuye en un factor $\sqrt{2}$:

$$S \equiv \frac{c \Delta t}{\Delta x} \leq \frac{1}{\sqrt{2}} \approx 0.707. \quad (156)$$

- **Advección, upwind 2D.** La condición práctica es

$$\Delta t \leq \frac{\Delta x}{\max(|u| + |v|)}, \quad (157)$$

que garantiza que la información numérica no recorra más de una celda por paso aun cuando la velocidad apunte en diagonal.

Regla operativa. En todos los códigos de esta semana el paso de tiempo se *calcula* a partir del criterio de estabilidad con un margen de seguridad (factores 0.4–0.5), nunca se fija a mano. Esta práctica elimina de raíz la clase de error más común en simulaciones explícitas: un Δt válido para una malla gruesa que se vuelve inestable al refinar.

5.4. Geometrías complejas sobre mallas cartesianas: máscaras

5.4.1. Máscara booleana y frontera escalonada

Un dominio Ω de forma arbitraria se representa con un arreglo booleano $M_{i,j} = [(x_i, y_j) \in \Omega]$. Las celdas con $M = \text{False}$ no participan en la evolución; las celdas activas con algún vecino inactivo forman la *frontera escalonada* (*staircase boundary*), que aproxima el contorno real con un error geométrico $\mathcal{O}(h)$. Esta es la limitación central del método: aunque el esquema interior sea de segundo orden, la frontera escalonada degrada la convergencia global cerca del contorno. Para geometrías curvas con requisitos

de precisión altos, la solución definitiva es el método de elementos finitos con mallas adaptadas al contorno (Semanas 8 y 9); la máscara es la herramienta de prototipado rápido que permite obtener respuestas físicamente correctas en minutos.

5.4.2. Condiciones de frontera sobre la máscara

Sobre la frontera escalonada se implementan las tres condiciones clásicas:

- **Dirichlet** (u prescrita): se fija el valor de las celdas frontera, o el de las celdas inactivas vecinas.
- **Neumann homogénea** (flujo nulo): se anulan los coeficientes de las caras que cruzan la frontera; el término diagonal cuenta únicamente los vecinos activos. Es la implementación natural en el ensamblaje disperso del Ejemplo 3.
- **Robin** (convección): se agrega un sumidero superficial linealizado; en formulación volumétrica, una celda frontera de tamaño h recibe el término $-h_{cv}(u - u_{\infty})/h$ por unidad de volumen.

5.4.3. Interfaces entre materiales: media armónica

Cuando el dominio contiene dos materiales con conductividades k_1 y k_2 , la conductividad en la cara entre dos celdas debe evaluarse con la *media armónica*

$$k_{\text{cara}} = \frac{2k_1k_2}{k_1 + k_2}, \quad (158)$$

y no con la media aritmética. La razón es física: las dos medias celdas actúan como resistencias térmicas *en serie*, y la media armónica es la única que garantiza la continuidad del flujo $q = -k \partial T / \partial n$ a través de la interfaz. Con contraste fuerte ($k_2/k_1 \approx 667$ en el Ejemplo 1), la media aritmética sobreestima el acoplamiento térmico en órdenes de magnitud.

5.5. Ejemplo 1 — Difusión 2D transitoria en una sección compuesta con generación interna

5.5.1. Planteamiento

Un componente de sección circular (radio exterior $R_{\text{ext}} = 6 \text{ mm}$) está formado por una corona metálica conductora y un núcleo polimérico de baja conductividad que aloja elementos sensibles. Durante un transitorio eléctrico de 0.3 s, la corona disipa calor Joule con densidad volumétrica $Q_J = 4 \times 10^8 \text{ W/m}^3$; después la fuente se apaga y el componente se enfría por convección natural. La pregunta de ingeniería es: ¿qué temperatura alcanza la interfaz con el núcleo y cuánto tarda el interior en enterarse del pulso?

El modelo es la ecuación de calor con propiedades variables por material:

$$\rho c_p(\mathbf{x}) \frac{\partial T}{\partial t} = \nabla \cdot (k(\mathbf{x}) \nabla T) + Q(\mathbf{x}, t), \quad \mathbf{x} \in \Omega \text{ (disco)}, \quad (159)$$

con condición inicial uniforme $T_0 = 25^\circ\text{C}$, frontera exterior adiabática con sumidero convectivo ($h_{cv} = 25 \text{ W}/(\text{m}^2 \text{ K})$) y fuente conmutada: $Q = Q_J$ en la corona si $t < t_{\text{pulso}}$ y $Q = 0$ en otro caso.

Tabla 23: Parámetros del Ejemplo 1. El contraste de conductividades entre corona y núcleo es de tres órdenes de magnitud.

Propiedad	Símbolo	Núcleo	Corona
Conductividad [$\text{W}/(\text{m K})$]	k	0.30	200
Densidad [kg/m^3]	ρ	1400	2700
Calor específico [$\text{J}/(\text{kg K})$]	c_p	1200	900
Radio [mm]	R	1.8	6.0

5.5.2. Discretización

El disco se embebe en una malla cartesiana de 121×121 nodos ($\Delta x = 0.105 \text{ mm}$) mediante una máscara circular. Las conductividades de cara se evalúan con la media armónica (Ec. 158) y el paso de tiempo se calcula con el criterio FTCS 2D (Ec. 155) usando la difusividad máxima del dominio ($\alpha_{\text{máx}} = 8.23 \times 10^{-5} \text{ m}^2/\text{s}$, la de la corona), con margen 0.4: $\Delta t = 13.4 \mu\text{s}$, es decir 44 792 pasos para cubrir 0.6 s.

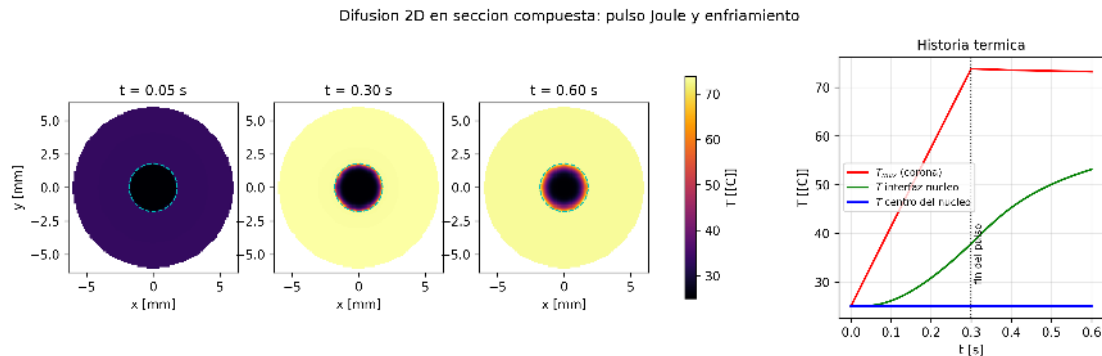


Figura 33: Difusión 2D en la sección compuesta. Paneles 1–3: campo de temperatura durante el pulso ($t = 0.05 \text{ s}$), al final del pulso ($t = 0.30 \text{ s}$) y tras el enfriamiento parcial ($t = 0.60 \text{ s}$); la circunferencia punteada marca la interfaz corona–núcleo. Panel derecho: historia térmica de la corona, de la interfaz del núcleo y del centro. Resultados del Código 1 del Anexo de esta semana.

5.5.3. Resultados y lectura metodológica

Tres observaciones cuantitativas estructuran la interpretación:

1. **La corona es casi isoterma.** Su temperatura máxima, 73.8°C , coincide con la estimación adiabática $T_0 + Q_J t_{\text{pulso}} / (\rho_2 c_{p,2}) \approx 74.4^\circ\text{C}$: la alta difusividad homogeneiza la corona en tiempos mucho menores que el pulso, y las pérdidas convectivas son despreciables en 0.6 s. Verificar un resultado 2D contra un balance 0D es la primera validación obligada de cualquier simulación térmica.
2. **El núcleo actúa como filtro pasa-bajas térmico.** La interfaz alcanza 53.1°C , pero el centro permanece en 25.0°C durante toda la simulación. La profundidad de penetración difusiva $\delta \sim \sqrt{\alpha_1 t} \approx 0.33 \text{ mm}$ para $t = 0.6 \text{ s}$ es mucho menor que el radio del núcleo (1.8 mm): el pulso simplemente no ha llegado al centro. El retardo térmico es una consecuencia directa de la EDP parabólica, no un artefacto.
3. **La media armónica importa.** Si se reemplaza por la media aritmética, la conductividad de la cara en la interfaz pasa de $0.6 \text{ W}/(\text{m K})$ a $100 \text{ W}/(\text{m K})$, y el núcleo se calienta espuriamente dos órdenes de magnitud más rápido. El error no es sutil: cambia la respuesta de ingeniería.

5.6. Ejemplo 2 — Onda 2D con obstáculo: reflexión y difracción

5.6.1. Planteamiento

Un pulso de presión se genera en un recinto cuadrado de $1 \text{ m} \times 1 \text{ m}$ (medio: aire, $c = 340 \text{ m/s}$) dividido por una pared rígida con una rendija de apertura $a = 0.10 \text{ m}$. El fenómeno gobernante es la ecuación de onda 2D:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad (160)$$

con $u = 0$ en la pared y en el contorno (fronteras rígidas), pulso gaussiano inicial de ancho $\sigma = 25 \text{ mm}$ y velocidad inicial nula. Este montaje reproduce el experimento clásico de difracción: detrás de la rendija el frente de onda se curva y la apertura actúa como fuente secundaria (principio de Huygens), un comportamiento común a ondas acústicas, mecánicas y ópticas.

5.6.2. Discretización

Se emplea el esquema *leapfrog*, centrado de segundo orden en tiempo y espacio:

$$u_{i,j}^{n+1} = 2u_{i,j}^n - u_{i,j}^{n-1} + (c \Delta t)^2 \nabla_h^2 u_{i,j}^n, \quad (161)$$

con arranque de Taylor $u^1 = u^0 + \frac{1}{2}(c \Delta t)^2 \nabla_h^2 u^0$ para preservar el segundo orden en el primer paso. La malla es de 401×401 nodos y el número de Courant se fija en $S = 0.5$, por debajo del límite 2D $1/\sqrt{2} \approx 0.707$ (Ec. 156), lo que da $\Delta t = 3.68 \mu\text{s}$. El obstáculo

se impone como condición de Dirichlet interna ($u = 0$ sobre la máscara de la pared) en cada paso.

Como diagnóstico de calidad se monitorea la energía discreta

$$E^n = \frac{1}{2} \sum_{i,j} \left[\left(\frac{u_{i,j}^{n+1} - u_{i,j}^{n-1}}{2\Delta t} \right)^2 + c^2 \|\nabla_h u_{i,j}^n\|^2 \right] \Delta x^2, \quad (162)$$

que para la ecuación de onda con fronteras rígidas es una cantidad conservada del problema continuo.

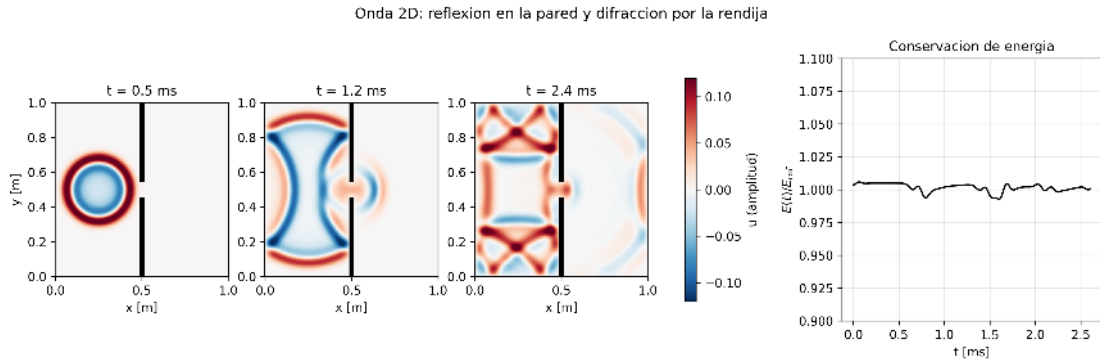


Figura 34: Onda 2D con obstáculo. Paneles 1–3: el pulso se expande ($t = 0.5$ ms), incide sobre la pared y la rendija emite un frente circular difractado ($t = 1.2$ ms), y el lado izquierdo desarrolla un patrón de reverberación mientras el frente difractado avanza ($t = 2.4$ ms). Panel derecho: energía discreta normalizada; la deriva tras el transitorio inicial es del 0.06 %. Resultados del Código 2 del Anexo de esta semana.

5.6.3. Resultados y lectura metodológica

1. **La conservación de energía es el termómetro del esquema.** El leapfrog es no disipativo: la energía discreta se conserva con deriva del 0.06 % en 708 pasos. Si la energía creciera sistemáticamente, el esquema sería inestable; si decayera, sería disipativo. Monitorear un invariante físico es la forma más barata de verificación continua en problemas hiperbólicos.
2. **La difracción es información geométrica cuantitativa.** El frente transmitido es aproximadamente circular y centrado en la rendija, con apertura angular controlada por la razón a/λ entre la apertura y la longitud de onda dominante del pulso. Reducir a ensancha el lóbulo difractado: el sistema se comporta como un filtro espacial.
3. **Costo del límite CFL 2D.** Trabajar a $S = 0.5$ en vez del límite 0.707 incrementa el número de pasos en ~ 41 %; es el precio de un margen de seguridad ante la frontera escalonada del obstáculo, donde el análisis de von Neumann (que supone dominio periódico homogéneo) ya no es estrictamente válido.

5.7. Ejemplo 3 — Poisson 2D en una placa perforada: el papel decisivo de las fronteras internas

5.7.1. Planteamiento

Una placa de concreto de $3\text{ m} \times 3\text{ m}$ genera calor de hidratación con densidad $q_h = 250\text{ W/m}^3$ durante el fraguado. Se comparan tres configuraciones: la placa sólida y una placa aligerada con un arreglo de 4×4 cavidades cuadradas de 0.45 m de lado (fracción de volumen resultante: 65.1%), considerando las cavidades en dos escenarios físicos: *selladas* (aire confinado, aproximadamente adiabáticas) o *ventiladas* (aire circulando a temperatura ambiente). El campo estacionario satisface la ecuación de Poisson

$$-k \nabla^2 T = q_h \quad \text{en el concreto,} \quad T = T_{\text{amb}} \quad \text{en el contorno exterior,} \quad (163)$$

con $k = 1.8\text{ W/(m K)}$, $T_{\text{amb}} = 22^\circ\text{C}$, y en las paredes de las cavidades: $\frac{\partial T}{\partial n} = 0$ (selladas) o $T = T_{\text{amb}}$ (ventiladas).

5.7.2. Discretización: ensamblaje disperso sobre el dominio enmascarado

Las incógnitas se numeran únicamente sobre las celdas activas mediante un mapa $(i, j) \mapsto p$; la matriz se ensambla en formato LIL y se convierte a CSR para resolver con `spsolve`. La condición de Neumann en las cavidades se implementa de forma natural: las caras que cruzan hacia un hueco simplemente no aportan conexión, y el coeficiente diagonal cuenta solo los vecinos activos. Para la malla de 181×181 , el caso aligerado tiene 20 592 incógnitas y 100 532 entradas no nulas: una densidad del 0.024% , que justifica plenamente la Tabla 22.

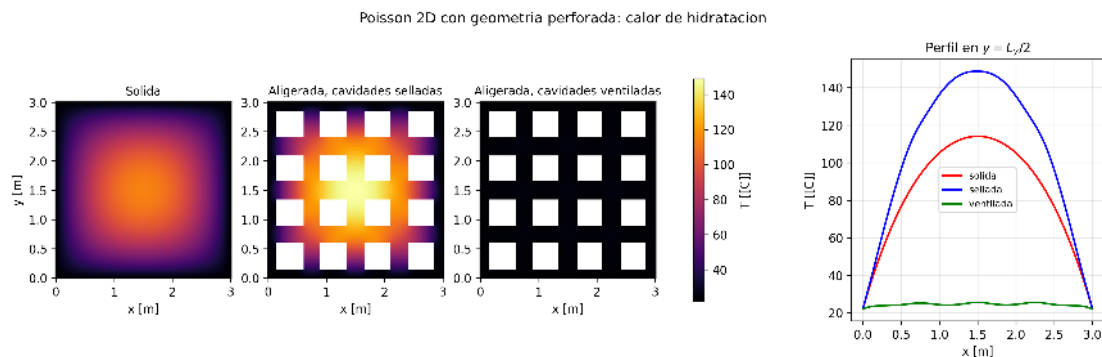


Figura 35: Calor de hidratación en la placa: sólida, aligerada con cavidades selladas y aligerada con cavidades ventiladas, bajo la misma escala de color. Panel derecho: perfil de temperatura por la línea media. La condición de frontera en las cavidades invierte la conclusión del estudio. Resultados del Código 3 del Anexo de esta semana.

5.7.3. Resultados y lectura metodológica

Tabla 24: Temperatura máxima según la configuración y la condición de frontera en las cavidades.

Configuración	$T_{\text{máx}} [^{\circ}\text{C}]$	Sobrecalentamiento
		$T_{\text{máx}} - T_{\text{amb}}$
Sólida	114.1	92.1 K
Aligerada, cavidades selladas	148.8	126.8 K
Aligerada, cavidades ventiladas	25.6	3.6 K

1. **La intuición ingenieril puede fallar.** Aligerar la placa reduce el volumen que genera calor (-35%), pero con cavidades selladas la temperatura máxima *sube* de 114.1°C a 148.8°C : las cavidades adiabáticas bloquean los caminos de conducción hacia el contorno y el calor restante queda atrapado en una red de nervaduras de mayor resistencia térmica.
2. **La condición de frontera interna decide el signo de la conclusión.** Con cavidades ventiladas a T_{amb} , el sobrecalentamiento colapsa a 3.6 K: cada cavidad se convierte en un sumidero interior y ninguna nervadura queda a más de ~ 0.1 m de una frontera fría. Entre los dos escenarios extremos hay un factor de 35 en el sobrecalentamiento. En un reporte doctoral, la condición de frontera asumida en cada superficie interna debe declararse y justificarse con el mismo rigor que la EDP.
3. **Los extremos acotan la realidad.** El caso real (cavidades con aire quieto y algo de convección natural) está entre ambos; los dos cálculos baratos proporcionan cotas rigurosas, una estrategia de modelado más defendible que un único cálculo con parámetros inciertos.

5.8. Ejemplo 4 — Convección-difusión 2D en un flujo recirculante: el número de Péclet de malla

5.8.1. Planteamiento

Un trazador pasivo (colorante, calor, contaminante) se libera en una cavidad cuadrada de $L = 0.10$ m donde el fluido recircula en un vórtice de una celda con campo de velocidad solenoidal analítico:

$$u = U_0 \sin \frac{\pi x}{L} \cos \frac{\pi y}{L}, \quad v = -U_0 \cos \frac{\pi x}{L} \sin \frac{\pi y}{L}, \quad (164)$$

con $U_0 = 0.05$ m/s y $\nabla \cdot \mathbf{u} = 0$ exactamente (verificado discretamente a nivel 10^{-14}). La concentración obedece

$$\frac{\partial c}{\partial t} + u \frac{\partial c}{\partial x} + v \frac{\partial c}{\partial y} = D \left(\frac{\partial^2 c}{\partial x^2} + \frac{\partial^2 c}{\partial y^2} \right). \quad (165)$$

Se comparan dos regímenes cambiando únicamente D : $Pe = U_0 L / D = 100$ y $Pe = 10\,000$.

5.8.2. Discretización y el parámetro que gobierna todo: Pe_h

La convección se discretiza con upwind de primer orden (seleccionando el nodo aguas arriba según el signo local de u y v) y la difusión con el stencil centrado. El paso de tiempo respeta simultáneamente las Ecs. 155 y 157. El parámetro de diagnóstico es el *número de Péclet de malla*

$$Pe_h = \frac{U_0 \Delta x}{D}, \quad (166)$$

que compara la convección con la difusión *a la escala de una celda*. El esquema upwind introduce una difusión numérica efectiva $D_{\text{num}} \approx U_0 \Delta x / 2$ (Semana 4); por tanto, $Pe_h \gg 2$ implica $D_{\text{num}} \gg D$ y la simulación deja de resolver la física de mezclado real.

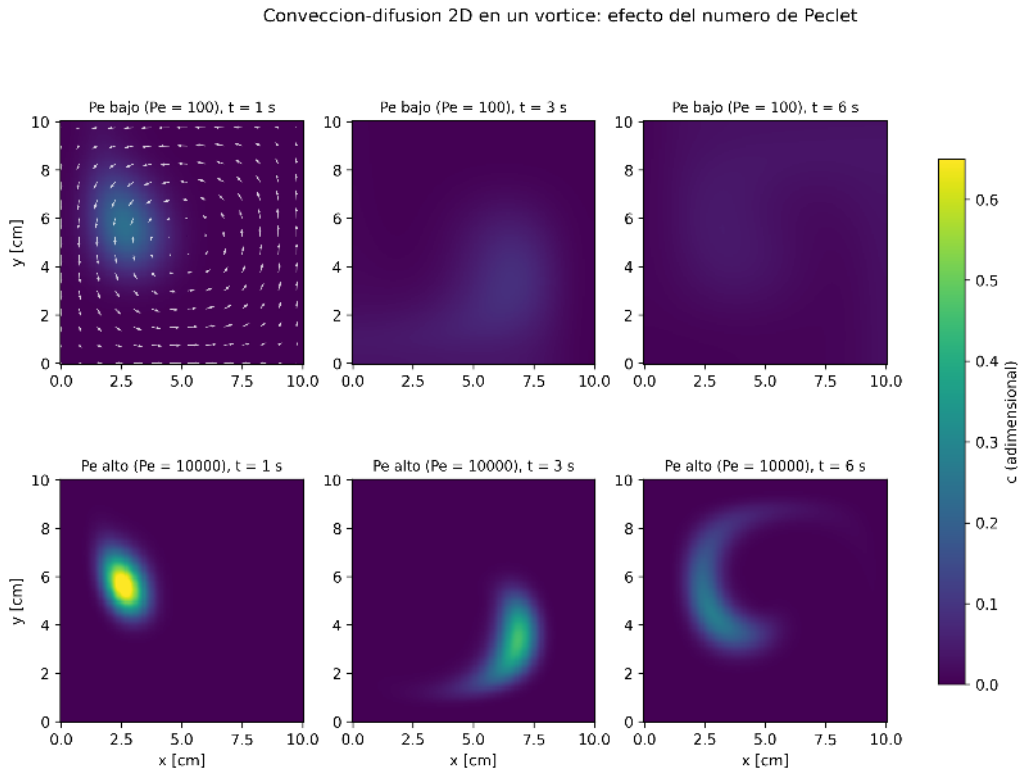


Figura 36: Trazador en el vórtice para dos números de Péclet, en tres instantes ($t = 1, 3$ y 6 s; flechas: campo de velocidad). Con $Pe = 100$ la mancha se difunde antes de completar una vuelta; con $Pe = 10\,000$ el vórtice estira la mancha en un filamento espiral, mecanismo cinemático que acelera el mezclado. Resultados del Código 4 del Anexo de esta semana.

5.8.3. Resultados y lectura metodológica

1. **Dos físicas distintas con la misma EDP.** Con $Pe = 100$ el pico de concentración cae a 0.043 en 6 s: difusión casi pura. Con $Pe = 10\,000$ el vórtice estira la mancha en filamentos cuyo espesor decrece con el tiempo; la difusión actúa entonces sobre gradientes cada vez más intensos. Este acoplamiento estiramiento–difusión es el mecanismo fundamental del mezclado en flujos laminares.
2. **El diagnóstico Pe_h revela un resultado contaminado.** Para $Pe = 10\,000$ con esta malla, $Pe_h = 62.5$ y $D_{\text{num}} \approx 1.6 \times 10^{-5} \text{ m}^2/\text{s}$, es decir *31 veces* la difusividad física. El campo del renglón inferior de la Figura 36 es cualitativamente correcto pero cuantitativamente dominado por difusión artificial: el pico final (0.273) está fuertemente subestimado respecto a la física real. Reportar Pe_h junto con Pe es una obligación de transparencia metodológica.
3. **La conservación de masa como métrica de verificación.** La forma no conservativa del término convectivo más el tratamiento de las fronteras producen una pérdida de masa del 1.75 % ($Pe = 100$) y 7.3 % ($Pe = 10\,000$) en 6 s. Una formulación en forma de flujo (volúmenes finitos) conservaría masa a precisión de máquina; cuantificar este defecto es parte del reporte, no un detalle omisible.

5.9. Ejemplo 5 — Advección pura bajo rotación sólida: disipación contra dispersión

5.9.1. Planteamiento

Cuando el campo de velocidad se conoce punto a punto (por ejemplo, estimado a partir de secuencias de imágenes aéreas mediante flujo óptico), el transporte de un campo de densidad ρ —densidad vehicular, concentración de partículas, intensidad luminosa— es advección pura:

$$\frac{\partial \rho}{\partial t} + u \frac{\partial \rho}{\partial x} + v \frac{\partial \rho}{\partial y} = 0. \quad (167)$$

El banco de pruebas canónico es la *rotación sólida*: $u = -\omega(y - y_c)$, $v = \omega(x - x_c)$, con $\omega = 2\pi \text{ rad/s}$. Tras exactamente una revolución ($t = 1 \text{ s}$), la solución exacta coincide con la condición inicial; *todo* el error observado es, por construcción, error numérico. La condición inicial combina un cono suave (sensible a la pérdida de amplitud) y un bloque discontinuo (sensible a las oscilaciones espurias).

5.9.2. Esquemas comparados

Se enfrentan los dos arquetipos de la Semana 4, ahora en 2D: upwind de primer orden (disipativo) y MacCormack, el equivalente bidimensional de Lax–Wendroff construido

como predictor–corrector:

$$\tilde{\rho} = \rho^n - \Delta t (u \delta_x^+ \rho^n + v \delta_y^+ \rho^n), \quad (168)$$

$$\rho^{n+1} = \frac{1}{2} [\rho^n + \tilde{\rho} - \Delta t (u \delta_x^- \tilde{\rho} + v \delta_y^- \tilde{\rho})], \quad (169)$$

donde $\delta^\pm$ son diferencias hacia adelante y hacia atrás. Malla de 201×201 , CFL efectivo 0.40, 3 142 pasos por revolución.

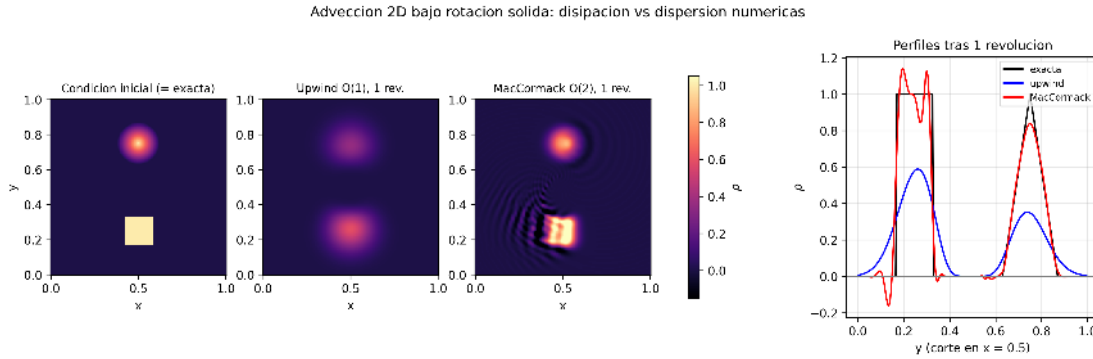


Figura 37: Rotación sólida de un cono suave y un bloque discontinuo tras una revolución completa. El upwind erosiona ambos perfiles (disipación); MacCormack preserva el cono pero genera oscilaciones y sobretiros alrededor del bloque (dispersión). Panel derecho: perfiles por el corte $x = 0.5$ contra la solución exacta. Resultados del Código 5 del Anexo de esta semana.

5.9.3. Resultados y lectura metodológica

Tabla 25: Métricas de error tras una revolución (valores exactos: pico = 1.000, mínimo = 0.000, masa = 100 %).

Esquema	Error L_2	Pico	Mínimo	Masa
Upwind $\mathcal{O}(1)$	0.1130	0.589	0.000	99.7 %
MacCormack $\mathcal{O}(2)$	0.0664	1.425	−0.448	100.0 %

1. **No existe el esquema lineal perfecto.** El upwind destruye el 41 % de la amplitud del pico pero nunca genera valores negativos (es monótono); MacCormack reduce el error L_2 a casi la mitad pero produce sobretiros del 42 % y densidades negativas de hasta −0.448, físicamente inadmisibles si ρ es una densidad. Esta disyuntiva es el contenido del teorema de Godunov: ningún esquema *lineal* de orden mayor que uno es monótono. Los esquemas modernos (limitadores de flujo, TVD, WENO) son no lineales precisamente para evadirlo.
2. **La métrica correcta depende del uso del modelo.** Si la cantidad de interés es la posición del máximo de densidad, MacCormack es superior; si el resultado alimenta un modelo posterior que exige positividad (una tasa, una probabilidad, un

control), el upwind es el único admisible de los dos. La selección de esquema es una decisión acoplada al uso downstream del resultado.

3. **El test con solución exacta es oro metodológico.** Diseñar un caso donde la respuesta correcta se conoce de antemano (aquí, por la simetría de la rotación) permite atribuir el cien por ciento del error al método. Todo capítulo de simulación de una tesis debería incluir al menos un test de este tipo.

5.10. Ejemplo 6 — Reconstrucción de un campo a partir de mediciones dispersas: interpolación armónica

5.10.1. Planteamiento

Una red de n_s sensores calibrados mide un campo escalar suave (temperatura, presión) en posiciones dispersas de una placa, con ruido de medición $\sigma = 0.3$ (en unidades del campo). Se requiere estimar el campo completo. Un estimador natural y computacionalmente idéntico a lo visto esta semana es el *interpolador armónico*: la función que satisface

$$\nabla^2 \hat{T} = 0 \text{ fuera de los sensores,} \quad \hat{T}(\mathbf{x}_s) = T_s^{\text{med}} \text{ en cada sensor,} \quad (170)$$

con Neumann homogénea en el contorno. Esta solución minimiza $\int_{\Omega} \|\nabla \hat{T}\|^2 d\Omega$ sujeta a las mediciones: es el campo *más suave posible* compatible con los datos, el análogo continuo de una membrana elástica clavada en los puntos medidos. Computacionalmente es el mismo ensamblaje disperso del Ejemplo 3, con los sensores actuando como condiciones de Dirichlet *puntuales interiores*.

5.10.2. Estudio de convergencia con la densidad de sensores

El campo verdadero se construye sintéticamente (suma de modos suaves no armónicos) para poder calcular el error exacto de reconstrucción. Se barre $n_s \in \{5, 10, 20, 40, 80, 160\}$ con posiciones aleatorias de semilla fija (reproducibilidad, Semana 1).

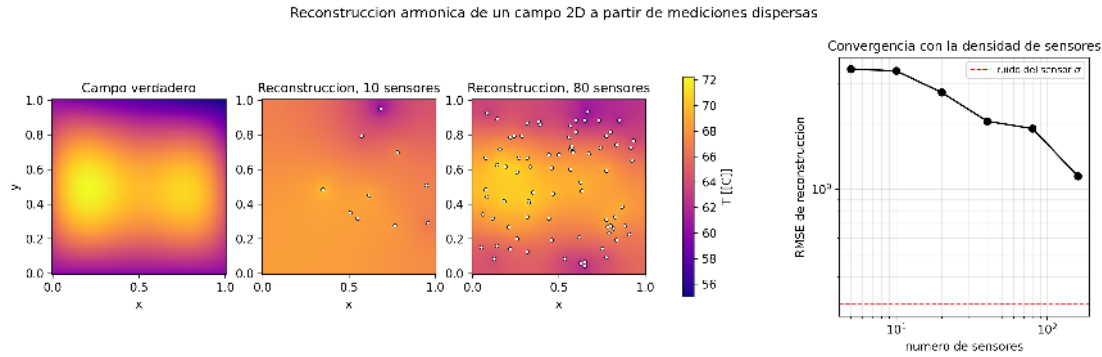


Figura 38: Reconstrucción armónica: campo verdadero, reconstrucciones con 10 y con 80 sensores (puntos blancos) y error RMS en función del número de sensores. La línea punteada marca el ruido de medición $\sigma = 0.3$. Resultados del Código 6 del Anexo de esta semana.

5.10.3. Resultados y lectura metodológica

1. **El error decrece, pero no hasta el ruido.** El RMSE cae de 3.51 con 5 sensores a 1.15 con 160, pero permanece muy por encima del piso de ruido $\sigma = 0.3$. La razón es estructural: el campo verdadero *no es armónico* ($\nabla^2 T_{\text{true}} \neq 0$), de modo que el interpolador tiene un *sesgo de modelo* irreducible que ningún número de sensores elimina por completo entre mediciones. Es la manifestación espacial del balance sesgo–varianza: el error total combina ruido de medición (varianza) y rigidez del modelo de interpolación (sesgo).
2. **La EDP como prior de regularización.** Imponer $\nabla^2 \hat{T} = 0$ equivale a declarar conocimiento previo (“el campo es suave”) que estabiliza un problema con muchas más incógnitas (121^2) que datos (n_s): un problema inverso en el sentido de la Semana 1, regularizado por la física. Si se conoce el término fuente, sustituir Laplace por Poisson reduce directamente el sesgo.
3. **El diseño de la red importa tanto como su tamaño.** Con posiciones aleatorias, regiones sin sensores cercanos al borde quedan pobremente estimadas (visible en la reconstrucción con 10 sensores). El error de reconstrucción como función de la posición es la herramienta cuantitativa para *diseñar* dónde colocar el siguiente sensor.

5.11. Visualización de campos escalares: reglas mínimas

La visualización en 2D no es cosmética: es el instrumento de lectura del resultado, y puede mentir. Reglas operativas usadas en todas las figuras de esta semana:

1. **Escala de color común** en todo conjunto de paneles que se compara entre sí (mis-mos $v_{\min}/v_{\max}$); de lo contrario la comparación visual es inválida. En la Figura 35,

la diferencia dramática entre configuraciones solo es legible porque las tres comparten escala.

2. **Mapas perceptualmente uniformes** (viridis, inferno, magma): la luminosidad varía monótonamente con el valor, de modo que gradientes iguales se perciben iguales. El clásico jet crea bandas artificiales y debe evitarse en material publicable.
3. **Mapas divergentes** (RdBu) solo para campos con cero físico significativo (la amplitud de onda de la Figura 34), centrando el blanco en cero.
4. **Un corte 1D acompaña al mapa 2D.** El perfil sobre una línea (Figuras 35 y 37) restituye la lectura cuantitativa que el mapa de color solo sugiere.
5. **El tiempo se muestra con instantáneas a tiempos idénticos entre casos** más una serie temporal de un escalar resumen ($T_{\text{máx}}(t)$, $E(t)$): las animaciones son útiles para explorar, pero el documento científico exige instantes comparables.

5.12. Síntesis de la semana

Tabla 26: Resumen de los seis ejemplos: familia de EDP, técnica central y lección metodológica.

Ej.	EDP	Técnica central	Lección metodológica
1	Parabólica	Máscara circular; media armónica en interfaces	Validar contra balance 0D; el retardo difusivo protege el núcleo
2	Hiperbólica	Leapfrog; CFL 2D $\leq 1/\sqrt{2}$	Monitorear un invariante físico (energía) como verificación
3	Elíptica	Ensamblaje disperso en dominio perforado	La condición de frontera interna puede invertir la conclusión
4	Conv.-difusión	Upwind 2D; Pe_h	Reportar Pe_h ; la difusión numérica puede dominar
5	Advección pura	Upwind vs MacCormack; test exacto	Disipación vs dispersión: teorema de Godunov
6	Elíptica (inversa)	Dirichlet puntual interior	La EDP como regularizador; sesgo de modelo vs ruido

La Semana 6 introducirá los métodos Monte Carlo: cuando la incertidumbre de los parámetros es parte esencial del problema, los campos deterministas de esta semana se convierten en distribuciones, y cada simulación 2D pasa a ser una muestra de un experimento estadístico.

Apéndice Semana 5. Referencias verificables

- A1. LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM. <https://doi.org/10.1137/1.9780898717839>

- A2. Courant, R., Friedrichs, K. y Lewy, H. (1928). Über die partiellen Differenzengleichungen der mathematischen Physik. *Mathematische Annalen*, 100, 32–74. <https://doi.org/10.1007/BF01448839>
- A3. Patankar, S. V. (1980). *Numerical Heat Transfer and Fluid Flow*. Hemisphere / CRC Press. <https://doi.org/10.1201/9781482234213>
- A4. MacCormack, R. W. (1969). The effect of viscosity in hypervelocity impact cratering. *AIAA Paper* 69–354. <https://doi.org/10.2514/6.1969-354>
- A5. Zalesak, S. T. (1979). Fully multidimensional flux-corrected transport algorithms for fluids. *Journal of Computational Physics*, 31(3), 335–362. [https://doi.org/10.1016/0021-9991\(79\)90051-2](https://doi.org/10.1016/0021-9991(79)90051-2)
- A6. Saad, Y. (2003). *Iterative Methods for Sparse Linear Systems* (2nd ed.). SIAM. <https://doi.org/10.1137/1.9780898718003>

5.13. Anexo Semana 5. Códigos para Google Colab

Propósito del anexo. Los seis códigos siguientes corresponden exactamente a las simulaciones cuyos resultados se discutieron en la sección principal. Cada código está completamente comentado y es autocontenido: basta copiarlo en una celda de Google Colab y ejecutarlo. Todos los parámetros físicos están agrupados en bloques claramente delimitados; modificar un parámetro y re-ejecutar la celda es la forma de realizar análisis de sensibilidad. Los archivos de figuras que generan estos scripts (.png, 300 dpi) son los mismos que aparecen en el documento.

5.13.1. Código 1 — Difusión 2D en sección compuesta (máscara circular y media armónica)

```

1      """
2      Semana 5 - Ejemplo 1: Difusion de calor 2D
3      transitoria en una seccion
4      transversal compuesta (dominio circular con dos
5      materiales)
6
7      Aplicacion generica: cualquier componente cilindrico
8      con generacion
9      interna de calor (un cable compuesto, un eje, una
10     barra de combustible).
11     La seccion contiene un nucleo de baja conductividad
12     (p.ej. un tubo
13     con fibras opticas) rodeado por una corona
14     conductora donde se

```

```

9         disipa calor por efecto Joule.
10
11         Fisica:  $\rho \cdot c_p \cdot \frac{dT}{dt} = \text{div}(k(x,y) \text{ grad } T) + Q(x, y)$ 
12
13         Esquema: FTCS 2D (stencil de 5 puntos) con
14         conductividad variable
15         evaluada con media armonica en las caras de celda.
16         Geometria: dominio circular embebido en una malla
17         cartesiana
18         mediante una mascara booleana (metodo de frontera
19         escalonada).
20         """
21
22         #
23         =====
24         # BLOQUE 1: Librerias
25         #
26         =====
27
28         import numpy as np
29         import matplotlib
30         matplotlib.use('Agg')
31         import matplotlib.pyplot as plt
32         import matplotlib as mpl
33
34         mpl.rcParams.update({
35             'font.size': 10, 'axes.titlesize': 11, 'axes.
36             labelsizsize': 10,
37             'figure.dpi': 110, 'savefig.dpi': 300,
38         })
39
40         #
41         =====
42         # BLOQUE 2: Parametros fisicos
43         # Dos materiales: nucleo (tubo polimerico) y corona
44         (aleacion Al)
45         #
46         =====
47
48         R_ext    = 6.0e-3          # [m]  radio exterior de la
49         seccion
50
51         R_nuc    = 1.8e-3          # [m]  radio del nucleo de
52         baja conductividad
53

```

```

39      # Material 1: nucleo polimerico (protege elementos
      sensibles)
40      k1, rho1, cp1 = 0.30, 1400.0, 1200.0      # [W/m/K], [
      kg/m3], [J/kg/K]
41      # Material 2: corona metalica (aluminio, donde se
      genera el calor)
42      k2, rho2, cp2 = 200.0, 2700.0, 900.0
43
44      # Generacion volumetrica de calor (efecto Joule)
      SOLO en la corona
45      Q_J = 4.0e8      # [W/m3] densidad de potencia
      durante el transitorio
46      t_pulso = 0.30      # [s] duracion del pulso de
      corriente
47      t_total = 0.60      # [s] tiempo total simulado
      (pulso + enfriamiento)
48
49      T0      = 25.0      # [C] temperatura inicial
      uniforme
50      h_cv      = 25.0      # [W/m2/K] conveccion natural
      en la superficie
51      T_inf = 25.0      # [C] temperatura ambiente
52
53      #
      =====
54      # BLOQUE 3: Malla cartesiana 2D y mascara del
      dominio circular
55      #
      =====
56      N      = 121      # nodos por
      direccion (malla N x N)
57      L      = 2.0 * R_ext * 1.05      # lado del
      cuadrado que contiene al circulo
58      dx = L / (N - 1)
59      x      = np.linspace(-L/2, L/2, N)
60      X, Y      = np.meshgrid(x, x, indexing='xy')
61      r      = np.sqrt(X**2 + Y**2)
62
63      dentro      = r <= R_ext      # mascara del
      dominio fisico
64      nucleo      = r <= R_nuc      # subdominio 1
65      corona      = dentro & ~nucleo      # subdominio 2
    
```

```

66         # frontera: celdas del dominio con al menos un
vecino fuera
67         front = dentro & (
68         ~np.roll(dentro, 1, 0) | ~np.roll(dentro, -1, 0) |
69         ~np.roll(dentro, 1, 1) | ~np.roll(dentro, -1, 1))
70
71         # Campos de propiedades por celda
72         k_map = np.where(nucleo, k1, k2)
73         rc_map = np.where(nucleo, rho1*cp1, rho2*cp2)
74         Q_map = np.where(corona, Q_J, 0.0)
75
76         #
=====
77         # BLOQUE 4: Paso de tiempo estable (criterio FTCS 2D
)
78         # dt <= dx^2 / (4 * alpha_max)
79         #
=====
80         alpha_max = (k_map / rc_map)[dentro].max()
81         dt = 0.40 * dx**2 / (4.0 * alpha_max)      # margen
de seguridad
82         n_pasos = int(np.ceil(t_total / dt))
83         print(f"dx = {dx*1e3:.3f} mm | alpha_max = {
alpha_max:.3e} m2/s")
84         print(f"dt = {dt*1e6:.2f} us | pasos = {n_pasos}")
85
86         #
=====
87         # BLOQUE 5: Conductividades de cara (media armonica)
88         # k_face = 2 k_i k_j / (k_i + k_j) garantiza
continuidad de flujo
89         # en la interfaz entre materiales
90         #
=====
91         def k_face(ka, kb):
92         return 2.0 * ka * kb / (ka + kb)
93
94         kE = k_face(k_map, np.roll(k_map, -1, 1))    # cara
este
95         kW = k_face(k_map, np.roll(k_map, 1, 1))     # cara
oeste

```

```

96         kN = k_face(k_map, np.roll(k_map, -1, 0))    # cara
norte
97         kS = k_face(k_map, np.roll(k_map, 1, 0))    # cara
sur
98
99         # Anular el flujo a traves de caras que cruzan la
frontera del circulo:
100         # equivale a imponer condicion de Neumann homogenea
(adiabatica) en la
101         # frontera escalonada. La perdida convectiva se
agrega como sumidero.
102         kE[~np.roll(dentro, -1, 1)] = 0.0
103         kW[~np.roll(dentro, 1, 1)] = 0.0
104         kN[~np.roll(dentro, -1, 0)] = 0.0
105         kS[~np.roll(dentro, 1, 0)] = 0.0
106
107         #
=====
108         # BLOQUE 6: Bucle temporal FTCS con fuente conmutada
109         # La conveccion superficial se modela como sumidero
volumetrico
110         # en las celdas de frontera: q_cv = h (T - T_inf) *
(P/A) ~ h(T-T_inf)/dx
111         #
=====
112         T = np.full((N, N), T0)
113         hist_t, hist_Tmax, hist_Tnuc = [], [], []
114         snaps = {}
115         t_snap = [0.05, 0.30, 0.60]
116         idx_c = N // 2                                # nodo central
117         j_int = idx_c + int(R_nuc/dx) - 2              # nodo dentro
del nucleo, cerca de la interfaz
118         hist_Tint = []
119
120         t = 0.0
121         for n in range(n_pasos):
122             Q_n = Q_map if t < t_pulso else np.zeros_like(Q_map)
123
124             TE = np.roll(T, -1, 1); TW = np.roll(T, 1, 1)
125             TN = np.roll(T, -1, 0); TS = np.roll(T, 1, 0)
126             lap = (kE*(TE-T) + kW*(TW-T) + kN*(TN-T) + kS*(TS-T)
) / dx**2

```

```

127
128         sumidero = np.zeros_like(T)
129         sumidero[front] = h_cv * (T[front] - T_inf) / dx    #
130         [W/m3]
131
132         T = T + dt * (lap + Q_n - sumidero) / rc_map
133         T[~dentro] = T0                                     # el exterior no
134         participa
135         t += dt
136
137         hist_t.append(t)
138         hist_Tmax.append(T[dentro].max())
139         hist_Tnuc.append(T[idx_c, idx_c])
140         hist_Tint.append(T[idx_c, j_int])
141         for ts in t_snap:
142             if ts not in snaps and t >= ts:
143                 snaps[ts] = T.copy()
144
145         print(f"T_max global          = {max(hist_Tmax):.1f} C"
146               )
147         print(f"T_max en el nucleo = {max(hist_Tnuc):.1f} C"
148               )
149         print(f"T_nucleo al final   = {hist_Tnuc[-1]:.1f} C")
150         print(f"T_max al final      = {hist_Tmax[-1]:.1f} C")
151         print(f"T interfaz nucleo (max) = {max(hist_Tint):.1
152               f} C")
153
154         #
155         =====
156         # BLOQUE 7: Visualizacion (campo escalar + historia
157         temporal)
158         #
159         =====
160
161         fig, axs = plt.subplots(1, 4, figsize=(15, 3.8),
162                                gridspec_kw={'width_ratios': [1, 1, 1, 1.25]})
163         vmax = max(hist_Tmax)
164         for ax, ts in zip(axs[:3], t_snap):
165             Tp = np.where(dentro, snaps[ts], np.nan)
166             im = ax.pcolormesh(X*1e3, Y*1e3, Tp, cmap='inferno',
167                               vmin=T0, vmax=vmax, shading='auto')
168             th = np.linspace(0, 2*np.pi, 200)

```

```

160         ax.plot(R_nuc*1e3*np.cos(th), R_nuc*1e3*np.sin(th),
161               'c--', lw=1)
162         ax.set_title(f"t = {ts:.2f} s")
163         ax.set_xlabel("x [mm]"); ax.set_aspect('equal')
164         axs[0].set_ylabel("y [mm]")
165         fig.colorbar(im, ax=axs[:3], shrink=0.85, label="T
166         [[C]]")
167
168         axs[3].plot(hist_t, hist_Tmax, 'r-', lw=1.6, label=r
169         "$T_{max}$ (corona)")
170         axs[3].plot(hist_t, hist_Tint, 'g-', lw=1.6, label=r
171         "$T$ interfaz nucleo")
172         axs[3].plot(hist_t, hist_Tnuc, 'b-', lw=1.6, label=r
173         "$T$ centro del nucleo")
174         axs[3].axvline(t_pulso, color='k', ls=':', lw=1)
175         axs[3].text(t_pulso*1.02, T0+5, "fin del pulso",
176         rotation=90, fontsize=8)
177         axs[3].set_xlabel("t [s]"); axs[3].set_ylabel("T [[C
178         ]])")
179
180         axs[3].set_title("Historia termica")
181         axs[3].legend(fontsize=8); axs[3].grid(alpha=0.3)
182
183         fig.suptitle("Difusion 2D en seccion compuesta:
184         pulso Joule y enfriamiento",
185         y=1.02)
186         plt.savefig('Figuras/fig_s5_ej1_difusion2D_cable.png
187         ',
188         dpi=300, bbox_inches='tight')
189         # plt.show()
190         print("Figura guardada.")
191
    
```

Listing 31: Semana 5 – Código 1: Difusión de calor 2D transitoria (FTCS, stencil de 5 puntos) en una sección circular de dos materiales con fuente Joule conmutada. Ejecutar en Google Colab.

5.13.2. Código 2 — Onda 2D con rendija (leapfrog y conservación de energía)

```

1         """
2         Semana 5 - Ejemplo 2: Ecuacion de onda 2D con
3         obstaculo --
4         reflexion y difraccion a traves de una rendija
    
```

```

5      Aplicacion generica: propagacion de perturbaciones
      mecanicas u
6      opticas en un medio bidimensional (ondas de presion
      generadas por
7      una descarga, vibraciones en una membrana, frentes
      de onda en
8      metrologia optica). El obstaculo con rendija
      reproduce el
9      experimento clasico de difraccion.
10
11     Fisica:  $d^2u/dt^2 = c^2 (d^2u/dx^2 + d^2u/dy^2)$ 
12     Esquema: leapfrog explicito de segundo orden en
      tiempo y espacio.
13     Estabilidad (CFL 2D):  $c*dt/dx \leq 1/\sqrt{2}$ .
14     """
15
16     #
17     =====
18     # BLOQUE 1: Librerias
19     #
20     =====
21     import numpy as np
22     import matplotlib
23     matplotlib.use('Agg')
24     import matplotlib.pyplot as plt
25     import matplotlib as mpl
26
27     mpl.rcParams.update({
28         'font.size': 10, 'axes.titlesize': 11, 'axes.
29         labelsize': 10,
30         'figure.dpi': 110, 'savefig.dpi': 300,
31     })
32
33     #
34     =====
35     # BLOQUE 2: Parametros fisicos y geometricos
36     #
37     =====
38     Lx, Ly = 1.0, 1.0      # [m]      dominio cuadrado
39     c       = 340.0        # [m/s]    velocidad de
      propagacion (aire)

```



```

35         T_fin = 2.6e-3           # [s]      tiempo total
simulado
36
37         # Fuente: pulso gaussiano inicial centrado en (x0,
y0)
38         x0, y0 = 0.25, 0.50     # [m]      posicion de la
fuente
39         sigma = 0.025           # [m]      ancho del pulso
40
41         # Obstaculo: pared vertical en x = x_b con una
rendija centrada
42         x_b = 0.50              # [m]      posicion de la
pared
43         espesor = 0.02          # [m]      espesor de la
pared
44         a_rendija = 0.10        # [m]      apertura de la
rendija
45
46         #
=====
47         # BLOQUE 3: Malla y numero de Courant 2D
48         #
=====
49
50         N = 401
51         dx = Lx / (N - 1)
52         x = np.linspace(0, Lx, N)
53         X, Y = np.meshgrid(x, x, indexing='xy')
54
55         # CFL 2D: elegimos  $S = c \, dt/dx = 0.5 < 1/\sqrt{2} \sim$ 
0.707
56
57         S = 0.5
58         dt = S * dx / c
59         n_pasos = int(np.ceil(T_fin / dt))
60         print(f"dx = {dx*1e3:.2f} mm | dt = {dt*1e6:.2f} us
| pasos = {n_pasos}")
61         print(f"Numero de Courant 2D: S = {S} (limite 1/sqrt
(2) = {1/np.sqrt(2):.3f})")
62
63         # Mascara del obstaculo (True = pared rigida, u = 0)
pared = (np.abs(X - x_b) <= espesor/2) & (np.abs(Y -
Ly/2) > a_rendija/2)

```

```

64      #
=====
65      # BLOQUE 4: Condiciones iniciales
66      # u^0: pulso gaussiano; u^1 obtenido con arranque de
        Taylor
67      # u^1 = u^0 + (dt^2 c^2 / 2) lap(u^0) (velocidad
        inicial nula)
68      #
=====
69      def laplaciano(u):
70          lap = np.zeros_like(u)
71          lap[1:-1, 1:-1] = (u[1:-1, 2:] + u[1:-1, :-2] +
72          u[2:, 1:-1] + u[:-2, 1:-1] -
73          4.0 * u[1:-1, 1:-1]) / dx**2
74          return lap
75
76          u0 = np.exp(-((X - x0)**2 + (Y - y0)**2) / (2 *
        sigma**2))
77          u0[pared] = 0.0
78          u1 = u0 + 0.5 * (c * dt)**2 * laplaciano(u0)
79          u1[pared] = 0.0
80
81      #
=====
82      # BLOQUE 5: Bucle temporal leapfrog
83      # u^{n+1} = 2u^n - u^{n-1} + (c dt)^2 lap(u^n)
84      # Fronteras exteriores: Dirichlet u = 0 (paredes
        rigidas)
85      # Energia discreta: E = sum( (du/dt)^2 + c^2 |grad u
        |^2 ) dx^2 / 2
86      #
=====
87      u_prev, u = u0.copy(), u1.copy()
88      snaps, hist_t, hist_E = {}, [], []
89      t_snap = [0.5e-3, 1.2e-3, 2.4e-3]
90
91      t = dt
92      for n in range(n_pasos):
93          u_next = 2.0*u - u_prev + (c*dt)**2 * laplaciano(u)
94          u_next[pared] = 0.0 # obstaculo rigido (
        Dirichlet interno)
95          u_next[0, :] = u_next[-1, :] = 0.0

```

```

96     u_next[:, 0] = u_next[:, -1] = 0.0
97
98     # Energia discreta (diagnostico de conservacion)
99     ut = (u_next - u_prev) / (2*dt)
100     ux = (u[1:-1, 2:] - u[1:-1, :-2]) / (2*dx)
101     uy = (u[2:, 1:-1] - u[:-2, 1:-1]) / (2*dx)
102     E = 0.5*np.sum(ut[1:-1, 1:-1]**2)*dx**2 \
103     + 0.5*c**2*(np.sum(ux**2) + np.sum(uy**2))*dx**2
104     hist_t.append(t); hist_E.append(E)
105
106     u_prev, u = u, u_next
107     t += dt
108     for ts in t_snap:
109         if ts not in snaps and t >= ts:
110             snaps[ts] = u.copy()
111
112     E = np.array(hist_E)
113     deriva = abs(E[-1] - E[len(E)//4]) / E[len(E)//4] *
100
114     print(f"Deriva de energia tras el transitorio
inicial: {deriva:.2f} %")
115
116     #
=====
117     # BLOQUE 6: Visualizacion
118     #
=====
119     fig, axs = plt.subplots(1, 4, figsize=(15.5, 3.9),
120     gridspec_kw={'width_ratios': [1, 1, 1, 1.2]})
121     for ax, ts in zip(axs[:3], t_snap):
122         up = np.ma.array(snaps[ts], mask=pared)
123         im = ax.pcolormesh(X, Y, up, cmap='RdBu_r', vmin
=-0.12, vmax=0.12,
124         shading='auto')
125         ax.add_patch(plt.Rectangle((x_b-espesor/2, 0),
espesor,
126         Ly/2 - a_rendija/2, color='k'))
127         ax.add_patch(plt.Rectangle((x_b-espesor/2, Ly/2 +
a_rendija/2),
128         espesor, Ly/2 - a_rendija/2, color='k'))
129         ax.set_title(f"t = {ts*1e3:.1f} ms")
130         ax.set_xlabel("x [m]"); ax.set_aspect('equal')

```

```

131         axs[0].set_ylabel("y [m]")
132         fig.colorbar(im, ax=axs[:3], shrink=0.85, label="u (
amplitud)")
133
134         axs[3].plot(np.array(hist_t)*1e3, E/E[len(E)//4], 'k
-', lw=1.4)
135         axs[3].set_xlabel("t [ms]")
136         axs[3].set_ylabel(r"$E(t)/E_{ref}$")
137         axs[3].set_title("Conservacion de energia")
138         axs[3].grid(alpha=0.3); axs[3].set_ylim(0.9, 1.1)
139
140         fig.suptitle("Onda 2D: reflexion en la pared y
difraccion por la rendija",
141                     y=1.03)
142         plt.savefig('Figuras/fig_s5_ej2_onda2D_difraccion.
png',
143                     dpi=300, bbox_inches='tight')
144         # plt.show()
145         print("Figura guardada.")
146

```

Listing 32: Semana 5 – Código 2: Ecuación de onda 2D con esquema leapfrog, obstáculo con rendija (difracción) y monitoreo de la energía discreta. Ejecutar en Google Colab.

5.13.3. Código 3 — Poisson 2D en placa perforada (ensamblaje disperso)

```

1         """
2         Semana 5 - Ejemplo 3: Ecuacion de Poisson 2D en un
dominio con
3         perforaciones -- calor de hidratacion en una losa
aligerada
4
5         Aplicacion generica: elementos estructurales de
concreto con
6         cavidades (losas reticulares o aligeradas). Durante
el fraguado, la
7         hidratacion del cemento genera calor; el campo
estacionario de
8         temperatura satisface la ecuacion de Poisson. Las
cavidades reducen
9         el volumen que genera calor y modifican la geometria
del dominio.

```

```

10
11     Fisica:       $-k * \text{lap}(T) = q_h$            en el concreto
12     T = T_amb           en el contorno exterior (
Dirichlet)
13     dT/dn = 0           en las paredes de las
cavidades
14     (Neumann homogenea, aislamiento aprox.)
15     Metodo:   ensamblaje disperso (scipy.sparse) del
stencil de 5 puntos
16     SOLO sobre las celdas activas del dominio
enmascarado.
17     """
18
19     #
=====
20     # BLOQUE 1: Librerias
21     #
=====
22     import numpy as np
23     import matplotlib
24     matplotlib.use('Agg')
25     import matplotlib.pyplot as plt
26     import matplotlib as mpl
27     from scipy.sparse import lil_matrix, csr_matrix
28     from scipy.sparse.linalg import spsolve
29
30     mpl.rcParams.update({
31         'font.size': 10, 'axes.titlesize': 11, 'axes.
labelsize': 10,
32         'figure.dpi': 110, 'savefig.dpi': 300,
33     })
34
35     #
=====
36     # BLOQUE 2: Parametros fisicos
37     #
=====
38     Lx, Ly = 3.0, 3.0           # [m]           planta de la losa
(vista en planta)
39     k_c      = 1.8              # [W/m/K]        conductividad del
concreto

```

```

40         q_h      = 250.0          # [W/m3]    generacion por
        hidratacion (pico tipico)
41         T_amb    = 22.0          # [C]        temperatura del
        contorno
42
43         # Patron de cavidades: arreglo de nc x nc huecos
        cuadrados
44         nc        = 4             # huecos por direccion
45         l_hueco   = 0.45          # [m] lado de cada hueco
46
47         #
        =====
48         # BLOQUE 3: Malla y mascaras (solida vs aligerada)
49         #
        =====
50         N = 181
51         dx = Lx / (N - 1)
52         x = np.linspace(0, Lx, N)
53         X, Y = np.meshgrid(x, x, indexing='xy')
54
55         def mascara_aligerada():
56             """True = celda de concreto (activa)."""
57             act = np.ones((N, N), dtype=bool)
58             paso = Lx / nc
59             for ii in range(nc):
60                 for jj in range(nc):
61                     cx = (ii + 0.5) * paso
62                     cy = (jj + 0.5) * paso
63                     hueco = (np.abs(X - cx) <= l_hueco/2) & \
64                         (np.abs(Y - cy) <= l_hueco/2)
65                     act &= ~hueco
66             return act
67
68         masc_solida = np.ones((N, N), dtype=bool)
69         masc_aligerada = mascara_aligerada()
70         frac_vol = masc_aligerada.sum() / masc_solida.sum()
71         print(f"Fraccion de volumen de la losa aligerada: {
        frac_vol*100:.1f} %")
72
73         #
        =====
74         # BLOQUE 4: Ensamblaje disperso sobre celdas activas

```

```

75         # Numeramos las incognitas con un mapa (i,j) ->
        indice global.
76         # Dirichlet en el contorno exterior; Neumann
        homogenea en los
77         # bordes internos (simplemente se omite la conexi3n
        al hueco y
78         # el coeficiente diagonal cuenta solo los vecinos
        activos).
79         #
        =====
80         def resolver_poisson(activa, bc_hueco='neumann'):
81             """bc_hueco: 'neumann' (cavidad sellada, adiabatica)
            o
            'dirichlet' (cavidad ventilada a T_amb)."""
82             interior = activa.copy()
83             interior[0, :] = interior[-1, :] = False
84             interior[:, 0] = interior[:, -1] = False
85
86             idx = -np.ones((N, N), dtype=int)
87             nodos = np.argwhere(interior)
88             idx[interior] = np.arange(len(nodos))
89             n_inc = len(nodos)
90
91             A = lil_matrix((n_inc, n_inc))
92             b = np.full(n_inc, q_h / k_c * dx**2) # termino
93             fuente
94             vecinos = [(0, 1), (0, -1), (1, 0), (-1, 0)]
95
96             for p, (i, j) in enumerate(nodos):
97                 diag = 0.0
98                 for di, dj in vecinos:
99                     ii, jj = i + di, j + dj
100                     if not activa[ii, jj]:
101                         if bc_hueco == 'dirichlet':
102                             diag += 1.0
103                             b[p] += T_amb # cavidad ventilada a T_amb
104                             continue # 'neumann': flujo nulo
105                             diag += 1.0
106                             if interior[ii, jj]:
107                                 A[p, idx[ii, jj]] = -1.0
108                             else:

```

```

109         b[p] += T_amb                                # vecino Dirichlet
    exterior
110         A[p, p] = diag
111         A = csr_matrix(A)
112
113         T = np.full((N, N), np.nan)
114         T[interior] = spsolve(A, b)
115         T[activa & ~interior] = T_amb                # contorno
    exterior
116         return T, A, n_inc
117
118         T_sol, A_sol, n_sol = resolver_poisson(masc_solida)
119         T_ali, A_ali, n_ali = resolver_poisson(
120 masc_aligerada, 'neumann')
121         T_ven, A_ven, n_ven = resolver_poisson(
122 masc_aligerada, 'dirichlet')
123
124         print(f"Losa solida:      {n_sol} incognitas, nnz = {
125 A_sol.nnz}")
126         print(f"Losa aligerada: {n_ali} incognitas, nnz = {
127 A_ali.nnz}")
128         print(f"Densidad de A (aligerada): "
129 f"{A_ali.nnz / n_ali**2 * 100:.4f} % de entradas no
130 nulas")
131         print(f"T_max solida                                = {np.nanmax(
132 T_sol):.2f} C")
133         print(f"T_max aligerada (sellada)                  = {np.nanmax(
134 T_ali):.2f} C")
135         print(f"T_max aligerada (ventilada)                = {np.nanmax(
136 T_ven):.2f} C")
137
138         #
139         =====
140         # BLOQUE 5: Visualizacion comparativa
141         #
142         =====
143         fig, axs = plt.subplots(1, 4, figsize=(16, 4.0),
144 gridspec_kw={'width_ratios': [1, 1, 1, 1.15]})
145         vmax = np.nanmax(T_ali)
146         for ax, T, titulo in zip(
147 axs[:3], [T_sol, T_ali, T_ven],
148 ["Solida", "Aligerada, cavidades selladas",

```



```

139         "Aligerada, cavidades ventiladas"]):
140         im = ax.pcolormesh(X, Y, T, cmap='inferno', vmin=
T_amb, vmax=vmax,
141         shading='auto')
142         ax.set_title(titulo, fontsize=10)
143         ax.set_xlabel("x [m]"); ax.set_aspect('equal')
144         axs[0].set_ylabel("y [m]")
145         fig.colorbar(im, ax=axs[:3], shrink=0.85, label="T
[[C]]")
146
147         # Perfil por la linea media y = Ly/2
148         jm = N // 2
149         axs[3].plot(x, T_sol[jm, :], 'r-', lw=1.6, label="
solida")
150         axs[3].plot(x, T_ali[jm, :], 'b-', lw=1.6, label="
sellada")
151         axs[3].plot(x, T_ven[jm, :], 'g-', lw=1.6, label="
ventilada")
152         axs[3].set_xlabel("x [m]"); axs[3].set_ylabel("T [[C
]]")
153         axs[3].set_title(r"Perfil en $y = L_y/2$")
154         axs[3].legend(fontsize=8); axs[3].grid(alpha=0.3)
155
156         fig.suptitle("Poisson 2D con geometria perforada:
calor de hidratacion",
157         y=1.02)
158         plt.savefig('Figuras/fig_s5_ej3_poisson2D_losa.png',
159         dpi=300, bbox_inches='tight')
160         # plt.show()
161         print("Figura guardada.")
162
    
```

Listing 33: Semana 5 – Código 3: Ecuación de Poisson 2D sobre un dominio con cavidades, ensamblada en formato disperso (scipy.sparse) con condiciones Neumann o Dirichlet en las fronteras internas. Ejecutar en Google Colab.

5.13.4. Código 4 — Convección-difusión 2D en un vórtice (Péclet de malla)

```

1         """
2         Semana 5 - Ejemplo 4: Conveccion-difusion 2D de un
escalar pasivo
3         en un flujo recirculante (vortice)
    
```

```

4
5      Aplicacion generica: transporte de un trazador (
6      colorante, calor,
7      contaminante) en el seno de un fluido en
8      recirculacion, como ocurre
9      en camaras de mezclado, tanques agitados o el
10     interior de equipos de
11     bombeo. El campo de velocidad se impone
12     analiticamente (vortice de
13     una celda) y el escalar se transporta segun la EDP
14     de conveccion-
15     difusion.
16
17     Fisica:  $dc/dt + u \, dc/dx + v \, dc/dy = D (d^2c/dx^2 +$ 
18      $d^2c/dy^2)$ 
19     Campo de velocidad solenoidal ( $\text{div } u = 0$ ):
20      $u = U0 * \sin(\pi x/L) * \cos(\pi y/L)$ 
21      $v = -U0 * \cos(\pi x/L) * \sin(\pi y/L)$ 
22     Esquema: upwind de primer orden (conveccion) +
23     centrado (difusion).
24     Estabilidad:  $dt \leq \min( dx/(|u|+|v|), \, dx^2/(4D) )$ .
25     El numero de Peclet de malla  $Pe_h = |u| \, dx / D$ 
26     controla el regimen.
27     """
28
29     #
30
31     =====
32     # BLOQUE 1: Librerias
33     #
34     =====
35
36     import numpy as np
37     import matplotlib
38     matplotlib.use('Agg')
39     import matplotlib.pyplot as plt
40     import matplotlib as mpl
41
42     mpl.rcParams.update({
43         'font.size': 10, 'axes.titlesize': 11, 'axes.
44         labels size': 10,
45         'figure.dpi': 110, 'savefig.dpi': 300,
46     })

```

```

35         #
=====
36         # BLOQUE 2: Parametros fisicos
37         # Dos regimenes: dominado por difusion (Pe bajo) y
por
38         # conveccion (Pe alto). Solo cambia D.
39         #
=====
40         L   = 0.10           # [m]      lado de la cavidad
41         U0   = 0.05           # [m/s]    velocidad
caracteristica del vortice
42         T_fin = 6.0           # [s]      tiempo simulado (~3
vueltas del vortice)
43
44         casos = {
45             "Pe bajo": 5.0e-5,    # [m2/s]  D grande  -> Pe
~ 100
46             "Pe alto": 5.0e-7,    # [m2/s]  D pequeno -> Pe
~ 10000
47         }
48
49         # Mancha inicial de trazador: gaussiana descentrada
50         x0, y0, s0 = 0.05, 0.075, 0.006    # [m]
51
52         #
=====
53         # BLOQUE 3: Malla y campo de velocidad analitico
54         #
=====
55         N   = 161
56         dx  = L / (N - 1)
57         x    = np.linspace(0, L, N)
58         X, Y = np.meshgrid(x, x, indexing='xy')
59
60         U    = U0 * np.sin(np.pi * X / L) * np.cos(np.pi * Y /
L)
61         V    = -U0 * np.cos(np.pi * X / L) * np.sin(np.pi * Y /
L)
62         print(f"max |u| = {np.abs(U).max():.3f} m/s ; "
63               f"div u (maximo discreto) ~ "
64               f"{np.abs(np.gradient(U, dx, axis=1) + np.gradient(V
, dx, axis=0)).max():.2e}")
    
```

```

65
66      #
=====
67      # BLOQUE 4: Esquema upwind 2D conservando el signo
local
68      #
=====
69      def derivada_upwind(c, vel, eje):
70          """Derivada de primer orden contra la corriente."""
71          atras = (c - np.roll(c, 1, axis=eje)) / dx #
72          usa nodo aguas arriba si vel>0
              adelante = (np.roll(c, -1, axis=eje) - c) / dx #
73          si vel<0
              return np.where(vel > 0, atras, adelante)
74
75      def laplaciano(c):
76          return (np.roll(c, -1, 1) + np.roll(c, 1, 1) +
77                  np.roll(c, -1, 0) + np.roll(c, 1, 0) - 4*c) / dx**2
78
79      resultados = {}
80      for nombre, D in casos.items():
81          Pe = U0 * L / D
82          Pe_h = U0 * dx / D
83          dt_adv = dx / (np.abs(U).max() + np.abs(V).max())
84          dt_dif = dx**2 / (4.0 * D)
85          dt = 0.4 * min(dt_adv, dt_dif)
86          n_pasos = int(np.ceil(T_fin / dt))
87          D_num = U0 * dx / 2.0 # difusion numerica del
upwind 0(u dx/2)
88          print(f"\n{nombre}: Pe = {Pe:.0f} | Pe_h = {Pe_h:.1f}
} | "
89          f"dt = {dt*1e3:.2f} ms | pasos = {n_pasos}")
90          print(f"    D fisica = {D:.1e} m2/s | D numerica (
upwind) ~ {D_num:.1e} m2/s "
91          f"(razon D_num/D = {D_num/D:.1f})")
92
93          c = np.exp(-((X-x0)**2 + (Y-y0)**2) / (2*s0**2))
94          masa0 = c.sum() * dx**2
95          snaps, t = {}, 0.0
96          t_snap = [1.0, 3.0, 6.0]
97          for n in range(n_pasos):

```

```

98         adv = U * derivada_upwind(c, U, 1) + V *
derivada_upwind(c, V, 0)
99         c = c + dt * (-adv + D * laplaciano(c))
100         # Fronteras: pared impermeable (Neumann homogenea,
copia del vecino)
101         c[0, :], c[-1, :] = c[1, :], c[-2, :]
102         c[:, 0], c[:, -1] = c[:, 1], c[:, -2]
103         t += dt
104         for ts in t_snap:
105             if ts not in snaps and t >= ts:
106                 snaps[ts] = c.copy()
107                 masa_f = c.sum() * dx**2
108                 print(f"    conservacion de masa: {masa_f/masa0
*100:.2f} % | "
109                       f"c_max final = {c.max():.3f}")
110                 resultados[nombre] = (snaps, Pe)
111
112         #
=====
113         # BLOQUE 5: Visualizacion (2 regimenes x 3 tiempos)
114         #
=====
115         fig, axs = plt.subplots(2, 3, figsize=(12.5, 8.0))
116         for fila, (nombre, (snaps, Pe)) in enumerate(
resultados.items()):
117             for col, ts in enumerate([1.0, 3.0, 6.0]):
118                 ax = axs[fila, col]
119                 im = ax.pcolormesh(X*100, Y*100, snaps[ts], cmap='
viridis',
120                                   vmin=0, vmax=0.65, shading='auto')
121                 if fila == 0 and col == 0:
122                     sk = 12
123                     ax.quiver(X[:,sk], X[:,sk]*100, Y[:,sk], Y[:,sk]*100,
124                               U[:,sk], V[:,sk],
125                               color='w', scale=1.0, width=0.004, alpha=0.8)
126                     ax.set_title(f"{nombre} (Pe = {Pe:.0f}), t = {ts:.0f
} s",
127                                  fontsize=9.5)
128                     ax.set_aspect('equal')
129                     if fila == 1: ax.set_xlabel("x [cm]")
130                     if col == 0: ax.set_ylabel("y [cm]")

```

```

131         fig.colorbar(im, ax=axis, shrink=0.8, label="c (
adimensional)")
132         fig.suptitle("Conveccion-difusion 2D en un vortice:
efecto del numero de Peclet",
133                     y=0.98)
134         plt.savefig('Figuras/fig_s5_ej4_convdif2D_vortice.
png',
135                     dpi=300, bbox_inches='tight')
136         # plt.show()
137         print("\nFigura guardada.")
138

```

Listing 34: Semana 5 – Código 4: Transporte convectivo-difusivo de un trazador en un vórtice analítico, esquema upwind 2D, con diagnóstico del número de Péclet de malla y de la difusión numérica. Ejecutar en Google Colab.

5.13.5. Código 5 — Advección 2D bajo rotación sólida (upwind vs MacCormack)

```

1      """
2      Semana 5 - Ejemplo 5: Adveccion pura 2D de un campo
de densidad bajo
3      rotacion solida -- difusion numerica vs dispersion
numerica
4
5      Aplicacion generica: transporte de un campo de
densidad (densidad
6      vehicular, concentracion de particulas, intensidad
en un flujo
7      optico) cuando el campo de velocidad se conoce en
cada punto, por
8      ejemplo estimado a partir de imagenes aereas. La
rotacion solida es
9      el banco de pruebas clasico: tras una revolucion
completa, la
10     solucion exacta coincide con la condicion inicial,
de modo que TODO
11     el error observado es error numerico.
12
13     Fisica:    d rho/dt + u d rho/dx + v d rho/dy = 0
14     Velocidad de rotacion solida alrededor del centro (
xc, yc):
15     u = -omega (y - yc) ;  v = omega (x - xc)

```

```

16         Esquemas comparados:
17         (a) upwind de primer orden -> disipativo (difusion
18         numerica)
19         (b) MacCormack (equivalente 2D de Lax-Wendroff, 2do
20         orden)
21         -> dispersivo (oscilaciones cerca de gradientes
22         fuertes)
23         """
24
25         #
26         =====
27         # BLOQUE 1: Librerias
28         #
29         =====
30
31         import numpy as np
32         import matplotlib
33         matplotlib.use('Agg')
34         import matplotlib.pyplot as plt
35         import matplotlib as mpl
36
37         mpl.rcParams.update({
38             'font.size': 10, 'axes.titlesize': 11, 'axes.
39             labelsizes': 10,
40             'figure.dpi': 110, 'savefig.dpi': 300,
41         })
42
43         #
44         =====
45         # BLOQUE 2: Parametros
46         #
47         =====
48
49         L      = 1.0                # [-] dominio unitario
50         omega = 2.0 * np.pi        # [rad/s]: una
51         revolucion por segundo
52         T_rev = 1.0                # [s] tiempo de una
53         revolucion completa
54         xc = yc = 0.5
55
56         N      = 201
57         dx = L / (N - 1)
58         x      = np.linspace(0, L, N)
59         X, Y = np.meshgrid(x, x, indexing='xy')

```

```

48
49     U = -omega * (Y - yc)
50     V =  omega * (X - xc)
51
52     # Paso de tiempo por CFL 2D
53     vmax = np.abs(U).max() + np.abs(V).max()
54     dt = 0.4 * dx / vmax
55     n_pasos = int(np.ceil(T_rev / dt))
56     dt = T_rev / n_pasos          # ajusta para cerrar
exactamente la revolucion
57     print(f"dx = {dx:.4f} | dt = {dt*1e3:.3f} ms | pasos
= {n_pasos}")
58     print(f"CFL efectivo = {vmax*dt/dx:.3f}")
59
60     #
=====
61     # BLOQUE 3: Condicion inicial -- cono suave + bloque
discontinuo
62     # El cono evalua la perdida de amplitud (disipacion)
;
63     # el bloque evalua oscilaciones espurias (dispersion
).
64     #
=====
65     def condicion_inicial():
66         rho = np.zeros((N, N))
67         # cono suave centrado en (0.5, 0.75)
68         r1 = np.sqrt((X-0.5)**2 + (Y-0.75)**2)
69         rho += np.maximum(0.0, 1.0 - r1/0.12)
70         # bloque cuadrado discontinuo centrado en (0.5,
0.25)
71         rho += np.where((np.abs(X-0.5) < 0.08) & (np.abs(Y
-0.25) < 0.08),
72             1.0, 0.0)
73         return rho
74
75         rho0 = condicion_inicial()
76
77         #
=====
78     # BLOQUE 4: Esquemas numericos

```



```

79      #
=====
80      def paso_upwind(r):
81          rx_b = (r - np.roll(r, 1, 1)) / dx
82          rx_f = (np.roll(r, -1, 1) - r) / dx
83          ry_b = (r - np.roll(r, 1, 0)) / dx
84          ry_f = (np.roll(r, -1, 0) - r) / dx
85          adv = U * np.where(U > 0, rx_b, rx_f) + V * np.where
(V > 0, ry_b, ry_f)
86          return r - dt * adv
87
88      def paso_maccormack(r):
89          # Predictor: diferencias hacia adelante
90          rx_f = (np.roll(r, -1, 1) - r) / dx
91          ry_f = (np.roll(r, -1, 0) - r) / dx
92          rp = r - dt * (U * rx_f + V * ry_f)
93          # Corrector: diferencias hacia atras sobre el
predictor
94          rx_b = (rp - np.roll(rp, 1, 1)) / dx
95          ry_b = (rp - np.roll(rp, 1, 0)) / dx
96          return 0.5 * (r + rp - dt * (U * rx_b + V * ry_b))
97
98      resultados = {}
99      for nombre, paso in [("Upwind 0(1)", paso_upwind),
100 ("MacCormack 0(2)", paso_maccormack)]:
101          r = rho0.copy()
102          for n in range(n_pasos):
103              r = paso(r)
104              r[0, :] = r[-1, :] = 0.0      # frontera lejana:
densidad nula
105              r[:, 0] = r[:, -1] = 0.0
106              err_L2 = np.sqrt(np.mean((r - rho0)**2))
107              pico = r.max()
108              minimo = r.min()
109              masa = r.sum() / rho0.sum() * 100
110              print(f"\n{n{nombre} tras 1 revolucion:")
111              print(f"    error L2 = {err_L2:.4f} | pico = {pico:.3
f} (exacto 1.000)")
112              print(f"    minimo = {minimo:.3f} (oscilaciones si <
0) | masa = {masa:.1f} %")
113              resultados[nombre] = r
114

```

```

115         #
=====
116         # BLOQUE 5: Visualizacion
117         #
=====
118         fig, axs = plt.subplots(1, 4, figsize=(16, 4.0),
119         gridspec_kw={'width_ratios': [1, 1, 1, 1.15]})
120         campos = [("Condicion inicial (= exacta)", rho0),
121         ("Upwind 0(1), 1 rev.", resultados["Upwind 0(1)"]),
122         ("MacCormack 0(2), 1 rev.", resultados["MacCormack 0
(2)"])]
123         for ax, (titulo, campo) in zip(axs[:3], campos):
124             im = ax.pcolormesh(X, Y, campo, cmap='magma', vmin
=-0.15, vmax=1.05,
125             shading='auto')
126             ax.set_title(titulo, fontsize=9.5)
127             ax.set_xlabel("x"); ax.set_aspect('equal')
128             axs[0].set_ylabel("y")
129             fig.colorbar(im, ax=axs[:3], shrink=0.85, label=r"$\
rho$")
130
131             # Perfiles por la linea x = 0.5
132             jm = N // 2
133             axs[3].plot(x, rho0[:, jm], 'k-', lw=1.4, label="
exacta")
134             axs[3].plot(x, resultados["Upwind 0(1)"][:, jm], 'b-
', lw=1.4,
135             label="upwind")
136             axs[3].plot(x, resultados["MacCormack 0(2)"][:, jm],
'r-', lw=1.4,
137             label="MacCormack")
138             axs[3].axhline(0, color='gray', lw=0.7)
139             axs[3].set_xlabel("y (corte en x = 0.5)"); axs[3].
set_ylabel(r"$\rho$")
140             axs[3].set_title("Perfiles tras 1 revolucion")
141             axs[3].legend(fontsize=8); axs[3].grid(alpha=0.3)
142
143             fig.suptitle("Adveccion 2D bajo rotacion solida:
disipacion vs dispersion "
144             "numericas", y=1.03)
145             plt.savefig('Figuras/fig_s5_ej5_adveccion2D_rotacion
.png',

```

```

146         dpi=300, bbox_inches='tight')
147     # plt.show()
148     print("\nFigura guardada.")
149

```

Listing 35: Semana 5 – Código 5: Test de rotación sólida con solución exacta conocida; comparación cuantitativa de disipación (upwind) y dispersión (MacCormack). Ejecutar en Google Colab.

5.13.6. Código 6 — Reconstrucción armónica desde sensores dispersos

```

1         """
2         Semana 5 - Ejemplo 6: Reconstruccion de un campo
3         escalar 2D a partir
4         de mediciones puntuales dispersas (interpolacion
5         armonica)
6
7         Aplicacion generica: una red de sensores calibrados
8         mide una variable
9         fisica (temperatura, presion, concentracion) en
10        puntos dispersos de
11        una placa o recinto, y se requiere estimar el campo
12        completo. Si el
13        campo verdadero es suave, una estimacion natural es
14        la funcion
15        ARMONICA que interpola las mediciones: se resuelve
16        la ecuacion de
17        Laplace con condiciones de Dirichlet puntuales en
18        los sensores.
19
20        Fisica/metodo:
21        lap(T) = 0                en todo nodo sin sensor
22        T = T_medida              en los nodos con sensor (
23        Dirichlet interno)
24        dT/dn = 0                 en el contorno exterior (Neumann
25        homogenea)
26
27        La solucion minimiza la integral de |grad T|^2
28        sujeta a las
29        mediciones: es el interpolador "de minima energia".
30
31        Estudio: error RMS de reconstruccion vs numero de
32        sensores.

```

```

20         """
21
22         #
23         =====
24         # BLOQUE 1: Librerías
25         #
26         =====
27
28         import numpy as np
29         import matplotlib
30         matplotlib.use('Agg')
31         import matplotlib.pyplot as plt
32         import matplotlib as mpl
33         from scipy.sparse import lil_matrix, csr_matrix
34         from scipy.sparse.linalg import spsolve
35
36         mpl.rcParams.update({
37             'font.size': 10, 'axes.titlesize': 11, 'axes.
38             labels.size': 10,
39             'figure.dpi': 110, 'savefig.dpi': 300,
40         })
41         rng = np.random.default_rng(7)      # semilla fija:
42         reproducibilidad
43
44         #
45         =====
46         # BLOQUE 2: Campo verdadero sintético (ground truth)
47         # Suma de modos suaves; representa el campo físico
48         real que la
49         # red de sensores intenta capturar.
50         #
51         =====
52
53         L = 1.0
54         N = 121
55         dx = L / (N - 1)
56         x = np.linspace(0, L, N)
57         X, Y = np.meshgrid(x, x, indexing='xy')
58
59         T_true = (60.0
60             + 18.0 * np.sin(np.pi * X) * np.sin(np.pi * Y)
61             + 7.0 * np.cos(2*np.pi * X) * np.sin(np.pi * Y)
62             - 5.0 * X * Y)

```

```

54         sigma_med = 0.3          # [unidades de T] ruido RMS de
    cada sensor calibrado

55
56         #
    =====
57         # BLOQUE 3: Reconstruccion armonica con sensores
    como Dirichlet
58         #
    =====

59         def reconstruir(n_sens):
60             # posiciones aleatorias (evitando una franja del
    borde)
61             ii = rng.integers(5, N-5, size=n_sens)
62             jj = rng.integers(5, N-5, size=n_sens)
63             medidas = T_true[ii, jj] + rng.normal(0.0, sigma_med
    , n_sens)

64
65             es_sensor = np.zeros((N, N), dtype=bool)
66             es_sensor[ii, jj] = True
67
68             idx = -np.ones((N, N), dtype=int)
69             libres = ~es_sensor
70             nodos = np.argwhere(libres)
71             idx[libres] = np.arange(len(nodos))
72             n_inc = len(nodos)
73
74             val_sensor = np.zeros((N, N))
75             val_sensor[ii, jj] = medidas
76
77             A = lil_matrix((n_inc, n_inc))
78             b = np.zeros(n_inc)
79             for p, (i, j) in enumerate(nodos):
80                 diag = 0.0
81                 for di, dj in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
82                     ni, nj = i + di, j + dj
83                     if ni < 0 or ni >= N or nj < 0 or nj >= N:
84                         continue          # Neumann: el borde no
    aporta flujo
85                 diag += 1.0
86                 if es_sensor[ni, nj]:
87                     b[p] += val_sensor[ni, nj]
88                 else:

```

```

89         A[p, idx[ni, nj]] = -1.0
90         A[p, p] = diag
91         A = csr_matrix(A)
92
93         T_rec = np.zeros((N, N))
94         T_rec[libres] = spsolve(A, b)
95         T_rec[es_sensor] = val_sensor[es_sensor]
96         rmse = np.sqrt(np.mean((T_rec - T_true)**2))
97         return T_rec, (ii, jj), rmse
98
99         # Barrido: numero de sensores
100        n_lista = [5, 10, 20, 40, 80, 160]
101        rmse_lista, casos = [], {}
102        for ns in n_lista:
103            T_rec, pos, rmse = reconstruir(ns)
104            rmse_lista.append(rmse)
105            casos[ns] = (T_rec, pos)
106            print(f"n_sensores = {ns:4d} | RMSE = {rmse:.3f}")
107            print(f"(ruido de sensor sigma = {sigma_med}: piso
de error esperado)")
108
109            #
=====
110            # BLOQUE 4: Visualizacion
111            #
=====
112            fig, axs = plt.subplots(1, 4, figsize=(16, 4.0),
113            gridspec_kw={'width_ratios': [1, 1, 1, 1.15]})
114            vmin, vmax = T_true.min(), T_true.max()
115
116            im = axs[0].pcolormesh(X, Y, T_true, cmap='plasma',
vmin=vmin, vmax=vmax,
117            shading='auto')
118            axs[0].set_title("Campo verdadero")
119            axs[0].set_xlabel("x"); axs[0].set_ylabel("y"); axs
[0].set_aspect('equal')
120
121            for ax, ns in zip(axs[1:3], [10, 80]):
122                T_rec, (ii, jj) = casos[ns]
123                ax.pcolormesh(X, Y, T_rec, cmap='plasma', vmin=vmin,
vmax=vmax,
124                shading='auto')

```

```

125         ax.plot(x[jj], x[ii], 'wo', ms=3, mec='k', mew=0.5)
126         ax.set_title(f"Reconstruccion, {ns} sensores")
127         ax.set_xlabel("x"); ax.set_aspect('equal')
128         fig.colorbar(im, ax=axes[:3], shrink=0.85, label="T
    [[C]])")
129
130         axes[3].loglog(n_lista, rmse_lista, 'ko-', lw=1.4)
131         axes[3].axhline(sigma_med, color='r', ls='--', lw=1,
132             label=r"ruido del sensor $\sigma$")
133         axes[3].set_xlabel("numero de sensores")
134         axes[3].set_ylabel("RMSE de reconstruccion")
135         axes[3].set_title("Convergencia con la densidad de
    sensores")
136         axes[3].legend(fontsize=8); axes[3].grid(alpha=0.3,
    which='both')
137
138         fig.suptitle("Reconstruccion armonica de un campo 2D
    a partir de "
139             "mediciones dispersas", y=1.03)
140         plt.savefig('Figuras/fig_s5_ej6_laplace_sensores.png
    ',
141             dpi=300, bbox_inches='tight')
142         # plt.show()
143         print("Figura guardada.")
144
    
```

Listing 36: Semana 5 – Código 6: Reconstrucción de un campo escalar 2D a partir de mediciones puntuales, resolviendo Laplace con condiciones de Dirichlet interiores; estudio RMSE vs número de sensores. Ejecutar en Google Colab.

5.14. Ejercicios Reto — Tarea Semana 5

Instrucciones generales. Los ejercicios están clasificados por nivel. Los Ejercicios 1 y 2 son obligatorios. El Ejercicio 3 es optativo y tiene valor de punto extra. En todos los casos se requiere: (a) formulación matemática explícita de la EDP, su dominio y sus condiciones de frontera (externas e internas); (b) código ejecutable en Google Colab; (c) mínimo una figura con ejes etiquetados, unidades, escala de color común cuando se comparen paneles, y título; y (d) interpretación física de los resultados en no menos de tres observaciones cuantitativas.

5.14.1. Ejercicio 1 — Límite CFL 2D y doble rendija (Nivel intermedio)

Modifica el Código 2 para explorar la estabilidad y la física de la difracción:

- Ejecuta la simulación con números de Courant $S \in \{0.70, 0.707, 0.72\}$. Reporta el instante en que la solución se vuelve inestable para $S = 0.72$ (criterio sugerido: primer paso en que $\max |u| > 10$) y grafica la energía discreta $E(t)$ para los tres casos en escala semilogarítmica.
- Sustituye la rendija única por una *doble rendija* (dos aperturas de 0.06 m separadas 0.20 m centro a centro). Grafica el campo en $t = 2.4$ ms y el perfil de $|u|$ a lo largo de la línea “pantalla” $x = 0.85$ m.
- Identifica en el perfil de la pantalla los máximos y mínimos de interferencia y compara cualitativamente su espaciamiento con la estimación $\Delta y \approx \lambda L_p / d$, donde d es la separación entre rendijas, L_p la distancia pared–pantalla y λ la longitud de onda dominante del pulso (estímalas del ancho σ de la gaussiana inicial).

Rúbrica.

- Inestabilidad localizada temporalmente con precisión, doble rendija implementada, patrón de interferencia medido en la pantalla y comparado con la estimación analítica.
- Inestabilidad demostrada y doble rendija implementada, pero sin análisis cuantitativo del patrón.
- Solo el inciso (a) completo.
- Código no ejecuta o resultados sin coherencia física.

5.14.2. Ejercicio 2 — Condición de Robin en las cavidades: del escenario sellado al ventilado (Nivel intermedio)

El Ejemplo 3 mostró dos extremos (Neumann y Dirichlet) en las paredes de las cavidades. El caso real es intermedio: convección con coeficiente h hacia aire a $T_\infty = T_{\text{amb}}$. Modifica el Código 3 para implementar la condición de Robin en las fronteras internas:

$$-k \frac{\partial T}{\partial n} \Big|_{\text{pared de cavidad}} = h (T - T_\infty). \quad (171)$$

Implementación sugerida. En el ensamblaje disperso, para una celda activa cuya cara cruza hacia un hueco, agrega al término diagonal $\beta = h \Delta x / k$ y al vector independiente βT_∞ (linealización estándar de la resistencia convectiva de cara; verifica que con $\beta \rightarrow 0$ se recupera Neumann y con $\beta \rightarrow \infty$ se recupera Dirichlet).

- Barre $h \in \{1, 5, 10, 50, 100\} \text{ W}/(\text{m}^2 \text{ K})$ y grafica $T_{\text{máx}}$ en función de h (escala semilogarítmica en h), incluyendo como líneas horizontales los tres valores de

referencia del Ejemplo 3 (sólida, sellada, ventilada).

- (b) Encuentra numéricamente el valor crítico h^* para el cual la placa aligerada deja de ser térmicamente *peor* que la sólida ($T_{\text{máx}}^{\text{alig}}(h^*) = T_{\text{máx}}^{\text{sólida}}$). Repórtalo con dos cifras significativas.
- (c) Discute: ¿qué mecanismo físico real (convección natural en cavidad cerrada, radiación, ventilación forzada) corresponde a cada orden de magnitud de h explorado?

Rúbrica.

- 3 Robin implementada y verificada en ambos límites, curva $T_{\text{máx}}(h)$ correcta y h^* reportado con justificación física.
- 2 Robin implementada y curva correcta, sin determinación de h^* o sin verificación de límites.
- 1 Planteamiento correcto de la discretización de Robin pero implementación incompleta.
- 0 Código no ejecuta o resultado sin coherencia física.

5.14.3. Ejercicio 3 — Esquema híbrido con limitador: evadir el teorema de Godunov (Nivel avanzado, punto extra)

El Ejemplo 5 mostró la disyuntiva disipación–dispersión de los esquemas lineales. Los esquemas modernos la evaden combinando *no linealmente* un esquema de orden alto con uno monótono. Implementa sobre el Código 5 un esquema híbrido simple por mezcla local:

$$\rho^{n+1} = \theta_{i,j} \rho_{\text{MC}}^{n+1} + (1 - \theta_{i,j}) \rho_{\text{UP}}^{n+1}, \quad (172)$$

donde ρ_{MC} y ρ_{UP} son los resultados de MacCormack y upwind en el mismo paso, y el peso local $\theta_{i,j} \in [0, 1]$ detecta la suavidad de la solución, por ejemplo

$$\theta_{i,j} = \max\left(0, 1 - \gamma \frac{|\rho_{i+1,j} - 2\rho_{i,j} + \rho_{i-1,j}| + |\rho_{i,j+1} - 2\rho_{i,j} + \rho_{i,j-1}|}{\max(\rho) - \min(\rho) + \epsilon}\right), \quad (173)$$

con γ un parámetro de sensibilidad y $\epsilon \approx 10^{-12}$.

- (a) Implementa el esquema híbrido y calibra γ para cumplir simultáneamente: pico del cono ≥ 0.80 , mínimo global ≥ -0.05 y error L_2 menor que el del upwind puro.
- (b) Reporta la tabla de métricas (error L_2 , pico, mínimo, masa) para upwind, MacCormack y el híbrido, tras una y tras tres revoluciones.
- (c) Grafica el mapa de $\theta_{i,j}$ en el instante final: ¿dónde activa el sensor el esquema monótono? Relaciona la respuesta con la posición del bloque discontinuo.

Rúbrica.

- 3 Híbrido funcional que cumple las tres metas, tabla completa a una y tres revoluciones, y mapa de θ interpretado.
- 2 Híbrido funcional que cumple dos de las tres metas, con tabla a una revolución.
- 1 Estructura del híbrido correcta (Ecs. 172 y 173 implementadas) pero sin calibración exitosa.